

GEOFICHAJE

GeoFichaje

Jesus David

Aplicacion web de control horario: el empleado ficha entrada o salida y el sistema comprueba DNI, ubicacion, horario y orden del fichaje.

37.8802566
-4.8040947

De que va el proyecto

GeoFichaje digitaliza el registro de jornada y evita fichajes fuera del centro o fuera de horario.

Control horario

Registra entradas y salidas de empleados para saber cuando empieza y termina cada jornada.

Geolocalizacion

El navegador obtiene la posicion del empleado y el backend comprueba que este cerca del centro.

Historial

Cada empleado ve sus registros, entradas acumuladas y tiempo trabajado calculado desde la base de datos.

Recorrido de un fichaje

La aplicacion se entiende mejor siguiendo el camino desde el empleado hasta MySQL.



- 01 El empleado entra con su DNI y solo accede si existe en la base de datos.
- 02 El navegador pide ubicacion GPS y activa los botones de entrada y salida.
- 03 El controlador recibe el formulario y delega las reglas en FichajeService.
- 04 Si todas las validaciones pasan, el registro se guarda en MySQL.

Datos que guarda

El proyecto no necesita muchas tablas: guarda empleados y sus registros de fichaje.

Empleado

tabla	empleados
campos	id, nombre, dni, email
clave	dni unico para login

Fichaje

tabla	registros_fichaje
relacion	ManyToOne con Empleado
estado	ENTRADA o SALIDA
GPS	Coordenadas embebidas: latitud, longitud

Pantallas principales

La interfaz tiene dos momentos: acceso por DNI y panel personal de fichaje.

1. Acceso

index.html pide el DNI y muestra error si el empleado no existe.

2. Panel

panel.html saluda al empleado y muestra resumen, mapa, botones e historial.

3. Ubicacion

JavaScript guarda latitud y longitud en campos ocultos antes de enviar el fichaje.

4. Respuesta

La misma pantalla muestra exito o el motivo exacto por el que no se permite fichar.

Reglas de negocio

La parte mas importante es decidir si el fichaje es valido antes de guardarlo.

Solo se guarda si pasa 3 filtros

Si uno falla, el servicio lanza un RuntimeException con un mensaje claro para el usuario.

Distancia

Haversine <= 200 m desde el centro

Horario

hora >= 6 && hora < 22

Orden

no entrada abierta; no salida sin entrada

Resultado

save(registroNuevo) solo si pasa todo

Resumen del empleado

El panel convierte los registros en informacion facil de entender.

Dias trabajados

Cuenta los registros cuyo estado es ENTRADA. En la interfaz aparece como entradas registradas.

Tiempo trabajado

Ordena entradas y salidas por fecha, empareja cada entrada con una salida y suma duraciones.

Codigo clave

El codigo esta organizado para que cada clase tenga una responsabilidad clara.

Controller

```
@PostMapping("/fichar")  
fichajeService.registrarFichaje(...)
```

Recibe DNI, latitud, longitud y estado. No guarda directamente: delega la logica al servicio.

Service

```
validarDistancia(...)  
validarHorario(...)  
validarOrdenDeFichaje(...)
```

Concentra las reglas importantes antes de crear el objeto Fichaje y llamar a save.

Repository

```
findByEmpleadoIdOrderByFechaHoraDesc(id)
```

Spring Data JPA crea la consulta automaticamente leyendo el nombre del metodo.

Vista

```
navigator.geolocation.getCurrentPosition(...)
```

panel.html obtiene la ubicacion del navegador y la envia al backend en campos ocultos.

Validacion por dentro

El ejemplo mas tecnico del proyecto es la comprobacion de distancia y orden de fichaje.

Distancia

Se usa la formula de Haversine para calcular metros entre la posicion del empleado y el centro de trabajo.

`distanciaEnMetros <= 200`

Orden

Busca el ultimo fichaje del empleado y evita casos incoherentes: dos entradas seguidas o salida sin entrada previa.

`ultimoRegistro.get().getEstado()`

Mejoras detectadas

Ademas de explicar lo que funciona, queda bien cerrar mostrando criterio tecnico.

Seguridad

no usar solo DNI como acceso

Tests

probar validaciones sin depender de MySQL

Config

unificar clave de Google Maps

Datos

mejorar emparejado entrada/salida

Ejemplo concreto

FichajeController lee app.google-maps-api-key, pero application.properties define key. Es una mejora sencilla y facil de defender.

Conclusion

GeoFichaje es un proyecto completo porque une interfaz, backend, base de datos y reglas reales.

El proyecto no solo guarda fichajes: decide si un fichaje tiene sentido.

Para el usuario

Acceso por DNI, botones de entrada/salida, mapa e historial personal.

Por dentro

Spring Boot separa controlador, servicios, repositorios y entidades JPA.

Valor del proyecto

Aplica reglas reales: cercanía al centro, horario permitido y orden correcto.